Causal Signal Credit Assignment for Verilog Code Generation via Counterfactual Hardware Verification

YALI DU, National Key Laboratory for Novel Software Technology, and the School of Artificial Intelligence, Nanjing University, China
MING LI*, National Key Laboratory for Novel Software Technology, and the School of Artificial Intelligence, Nanjing University, China

arXiv:2611.11848v1 [cs.AI] 9 Nov 2026

Large language models have improved Verilog generation, yet functional correctness remains difficult to optimize because simulation commonly provides a coarse module-level pass/fail outcome. Such feedback discards useful evidence from partially correct RTL modules and cannot determine whether a source statement is functionally responsible for an observed output behavior. We present a new method named Causal Signal Credit Assignment (CSCA), which is a counterfactual preference-optimization framework for Verilog generation. CSCA first performs continuous signal-level behavioral verification on sampled candidate modules. It then locates output-driving assignment spans and applies five interventions while holding the remaining module and testbench fixed. The resulting changes in signal-level errors are combined with behavioral agreement to form a continuous statement credit, which is mapped to response-token weights for strict weighted direct preference optimization. This enables CSCA to learn from locally supported statements without uniformly reinforcing an otherwise incorrect response. Built on Icarus Verilog and a CodeV-derived corpus, CSCA is evaluated on VerilogEval 2.0 and RTLLM v2.0, including both code completion and specification-to-RTL tasks. With a Qwen2.5-Coder-7B-Instruct backbone, CSCA leads the evaluated 7B-scale models and remains comparable to the much larger GPT-5.4-mini on the reported Verilog-generation benchmarks.

Additional Key Words and Phrases: Verilog Generation, Hardware Design Automation, Credit Assignment, Preference Optimization, Counterfactual Verification

ACM Reference Format:
Yali Du and Ming Li. 2026. Causal Signal Credit Assignment for Verilog Code Generation via Counterfactual Hardware Verification. 1, 1 (November 2026), 21 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Hardware description language (HDL) generation has emerged as a critical and high-value task in automated hardware design, enabling end-to-end synthesis of register-transfer-level (RTL) circuits from natural language or structural specifications. Automating reliable Verilog generation can substantially reduce manual engineering overhead, accelerate chip prototyping, and lower the barrier for hardware design education and lightweight IC development. Unlike general-purpose code generation, Verilog synthesis requires strict cycle-level functional correctness, precise timing semantics, and rigorous hardware interface compliance, making it far more challenging to optimize via data-driven learning. A useful Verilog module must satisfy an interface contract, compile under

*Corresponding author.

Authors' Contact Information: Yali Du, National Key Laboratory for Novel Software Technology, and the School of Artificial Intelligence, Nanjing University, China, duyl@lamda.nju.edu.cn; Ming Li, National Key Laboratory for Novel Software Technology, and the School of Artificial Intelligence, Nanjing University, China, lim@@lamda.nju.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM XXXX-XXXX/2026/11-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

an HDL toolchain, and exhibit the specified behavior over relevant input traces. Surface similarity to a reference implementation is insufficient: semantically different RTL expressions may implement the same circuit, while a single incorrect assignment can invalidate an otherwise plausible module.

Recent large language models (LLMs) have improved the quality of generated RTL, and benchmarks such as VerilogEval and RTLLM make functional evaluation accessible through compilation and simulation [3, 27, 29, 41]. Existing Verilog-specific approaches improve domain adaptation through curated instruction data, simulation-guided optimization, and preference learning from hardware feedback [28, 43, 53, 54]. These methods establish that compilation and simulation can provide substantially stronger supervision than text similarity alone. Signal-aware feedback further refines the module-level reward by retaining which outputs agree with the reference.

However, these techniques are hampered by a fundamental granularity mismatch between verification and model learning. Simulation-based evaluation typically returns a module-level outcome: a generated design either passes the available testbench or fails. This decision is appropriate for acceptance testing, but it is an inefficient training signal. The first limitation of generated RTL is frequently *partially correct*: one output may match the reference over all observed traces while another output, control condition, or state transition is incorrect. Treating both a partially correct module and a wholly incorrect module as equally negative discards behavioral evidence that could improve the generator. Yet the second limitation is that an output-level score still leaves an attribution question unresolved: *which statement should receive credit for that agreement?* A source assignment may be syntactically connected to an output but redundant over the observed traces; conversely, an apparently correct span may be compensated for by another statement or control path. Consequently, assigning preference weight from static association or output–statement co-occurrence can reinforce code that is correlated with, but not functionally responsible for, the rewarded behavior. This unresolved attribution gap motivates CSCA's counterfactual verification.

To tackle these two core limitations of existing Verilog generation methods, we introduce **Causal Signal Credit Assignment** (**CSCA**), a counterfactual training framework for Verilog generation. The core insight of CSCA is that RTL preference optimization should avoid uniformly rewarding or penalizing an entire generated RTL candidate. Instead, supervision should be localized to individual RTL statements, weighted by empirical functional evidence from both simulation and counterfactual perturbation. CSCA leverages this design principle to address the two core limitations identified above. (1) First, instead of discarding partially correct RTL modules by assigning a single binary pass/fail label at the module level, CSCA retains positive supervisory signals for functionally valid RTL statements even when the overall module contains unrelated bugs. (2) Second, the counterfactual intervention resolves the attribution gap: rather than naively attributing signal correctness to static syntactic co-occurrence, CSCA quantifies whether an RTL statement is functionally responsible for observed signal behavior by measuring how signal-level errors change upon local perturbation.

To operationalize this insight, CSCA adopts the following pipeline. CSCA first performs full-parameter supervised fine-tuning (SFT) on prompt–reference-code pairs to obtain a Verilog-capable generator. The SFT model samples several candidate RTL modules for each prompt. CSCA then executes every candidate and its reference implementation on the same testbench, retaining continuous correctness scores for individual output signals instead of collapsing the entire module into a binary label. For a statement that drives a target signal, CSCA constructs local Zero, One, Hold, Invert, and Delete variants, recompiles each valid variant, and measures the resulting changes in signal-level error counts. The combination of behavioral agreement and counterfactual degradation yields a continuous statement credit. CSCA translates these per-statement credits into token-level weights for Direct Preference Optimization (DPO). It constructs chosen/rejected pairs from sampled RTL candidates for identical prompts, and maps each credited RTL statement span to its corresponding

tokens in the model's generated response. This weighted DPO objective reinforces only those tokens associated with functionally validated statements, without propagating reward to unrelated, buggy code segments within the same candidate.

Built on Icarus Verilog and a CodeV-derived corpus, we evaluate CSCA on VerilogEval 2.0 and RTLLM v2.0, covering both code-completion and specification-to-RTL generation. All CSCA variants start from the same Qwen2.5-Coder-7B-Instruct backbone and use the same SFT initialization, rollout corpus, prompt format, decoding budget, and simulation harness. This controlled design isolates the effect of credit assignment from changes in backbone scale, training data, or verification infrastructure. Compared with the Qwen2.5-Coder-7B base model, CSCA improves greedy-decoding Pass@1 by 72.5% on VerilogEval Code Completion, 114.3% on Specification-to-RTL, and 38.9% on RTLLM v2.0, where each value is a relative improvement over the corresponding baseline. Against the strongest Verilog-specific baseline, QiMeng-SALV, the relative greedy Pass@1 gains are 3.5%, 6.7%, and 8.7% on the three tasks. Under matched stochastic sampling, the relative Pass@1/Pass@5 gains are 12.2%/2.0% on Code Completion, 8.5%/4.4% on Specification-to-RTL, and 14.6%/3.4% on RTLLM v2.0. Consequently, CSCA leads the evaluated 7B-scale models and remains competitive with the substantially larger GPT-5.4-mini on the reported Verilog-generation tasks. The ablation results attribute the gain to the combination of behavioral and counterfactual evidence. Additional analyses examine the mixing-coefficient sensitivity, intervention reliability, representative RTL-level corrections, and the observed costs of SFT, preference optimization, and inference.

In summary, our main contributions are summarized as follows:

- We identify the gap between module-level verification outcomes and statement-level learning signals for Verilog generation, and formulate CSCA as a framework that preserves reliable evidence from partially correct RTL modules.
- We introduce continuous signal-level behavioral verification and counterfactual causal attribution through five local RTL interventions. Their combined credit quantifies the empirical support for an output-driving statement under a fixed testbench.
- We develop strict statement-level weighted DPO, which maps counterfactually grounded statement credits to response-token weights, and conduct evaluation across VerilogEval 2.0 and RTLLM v2.0 together with component, sensitivity, reliability, and cost analyses.

2 Related Work

2.1 Verilog and RTL Code Generation

Before the advent of LLMs, hardware synthesis and high-level synthesis (HLS) focused on translating formal high-level specifications or behavioral descriptions into RTL, relying on hand-engineered compilers, rule-based rewriting, and constrained optimization [8, 30]. These tools excel at deterministic, correct-by-construction circuit translation but require rigorously structured input and cannot directly synthesize RTL from free-form natural-language specifications.

With the rise of LLMs, LLM-based RTL generation has progressed from early studies of whether pretrained code models can produce compilable Verilog to domain-adapted systems and standardized functional evaluation. Thakur et al. [41] introduced an early empirical study that fine-tuned code models on Verilog corpora and evaluated generated modules with compilation and testbenches. VerilogEval [27] and VerilogEval 2.0 [33] subsequently established widely used task formats and simulation-based functional evaluation protocols. RTLLM [29] broadens evaluation across arithmetic, control, memory, and miscellaneous RTL tasks. Moving beyond single-module problems, RTL-Repo [1] evaluates models in repository-scale RTL contexts, highlighting the importance of design context and multi-file dependencies.

A few works improve generation through domain adaptation or iterative hardware feedback. ChipNeMo [26] demonstrates the value of adapting LLMs to chip-design workflows. CodeV [54] constructs a large Verilog corpus and uses instruction tuning for RTL generation, while Origen [5] similarly studies hardware-oriented adaptation. AutoChip uses iterative feedback to refine generated designs [42], and RTLCoder applies simulation-derived functional rewards to RTL generation [28]. These efforts establish compilation and simulation as practical sources of learning and evaluation signals. CSCA builds on this setting but focuses on how verification evidence should be assigned within a partially correct candidate, rather than on a new backbone model or decoding procedure.

2.2 Verification-Guided RTL Learning and Counterfactual Testing

Verification-guided approaches use hardware execution to make generated RTL more functionally aligned with its specification. VeriSeek incorporates verification feedback into RTL generation [44], and VeriPrefer constructs preference data from hardware-verification outcomes [43]. QiMeng-SALV refines conventional module-level rewards with signal-level simulation feedback [53]. Such methods demonstrate that a module can provide useful supervision even when its overall verdict is negative.

CSCA is motivated by a remaining attribution gap. Signal agreement indicates that an output is correct on the observed tests, but it does not establish which source statement is responsible for the agreement. This distinction follows the broader causal perspective that observational association and the effect of an intervention are different objects [32]. CSCA operationalizes this distinction with executable RTL: it locally modifies an output-driving assignment and re-runs the same testbench to measure the resulting signal-level change.

The use of local program modifications is related to mutation testing, where mutants are used to assess the effectiveness of test suites [19]. The objective differs, however. Mutation testing asks whether a test suite distinguishes an original program from its mutants; CSCA uses behavior changes induced by syntax-preserving RTL interventions as continuous evidence for statement credit. The interventions are therefore not used to declare test adequacy or formal causality. Instead, they provide empirical, test-distribution-dependent evidence for selecting and weighting local preference supervision.

2.3 Execution Feedback, Testing, and Localized Program Reasoning

Recent software-engineering research increasingly treats execution as a useful source of evidence for code-model pipelines. In test generation, CodaMOSA combines LLM-produced tests with search-based exploration to escape coverage plateaus [23], while Fuzz4All uses LLMs as a generation and mutation engine for diverse formal and programming-language inputs [46]. Studies of LLM-based bug reproduction, directed test input generation, method-slicing-assisted test generation, and industrial acceptance testing further show both the value and the incompleteness of execution-derived evidence [20, 21, 45, 47]. These studies motivate the use of simulation in CSCA, but they generally use execution to create tests, improve coverage, or validate a complete candidate. CSCA instead uses a fixed testbench to assign continuous local credit inside a partially correct RTL candidate. Requirements-focused code-generation work, such as ClarifyGPT, is complementary: it improves the specification supplied to the generator, whereas CSCA improves the verification evidence used after generation [31].

Execution feedback has also been used to rank, refine, localize, and repair code. PairCoder iteratively refines candidate programs using execution feedback [51]; execution-based candidate ranking studies show that dynamic behavior can help separate plausible generated programs [38]; and LLM-based repair systems combine tests, program structure, or localization with generation [7, 15, 50, 52]. Unlike these approaches, CSCA does not seek a repaired final program or a single best

candidate. Its objective is to convert local behavioral evidence into statement-token preference weights during post-SFT optimization.

2.4 Evaluation, Robustness, and Efficient Code Generation

Recent SE benchmarks and empirical studies have also exposed the limits of single-score code-generation evaluation. JavaBench evaluates LLMs on object-oriented code generation [4], whereas UniTSyn studies large-scale data for LLM-assisted program testing [13]. Work on sketch-guided generation, retrieval-augmented preference tuning, context-aware generation, and excessive-token prevention addresses complementary challenges in structuring generation and controlling its cost [9, 12, 17, 24]. Robustness-oriented work such as AACEGEN further demonstrates that a code model can be sensitive to localized program changes [25]. These findings reinforce the need to distinguish a globally successful output from the local program decisions that are supported by executable evidence.

2.5 Preference Optimization and Reinforcement Learning

Preference optimization learns from relative judgments between candidate responses. Direct Preference Optimization (DPO) reformulates preference learning as a classification objective over chosen and rejected responses, without separately fitting a reward model [34]. Related policy-gradient methods, such as Proximal Policy Optimization (PPO), optimize a policy against a scalar reward while constraining policy updates [36]. These objectives are broadly applicable, but their behavior depends critically on how preference pairs or rewards are constructed.

For RTL generation, a preference label attached uniformly to a complete module can be noisy because a response may combine correct and incorrect statements. CSCA is complementary to DPO, PPO, and simulation-guided RL: it changes the construction and granularity of the supervision rather than replacing the underlying optimization interface. Specifically, it ranks sampled candidates with continuous behavioral and counterfactual credits, then maps credited RTL statement spans to token weights. This enables weighted DPO to emphasize functionally supported decisions while suppressing unrelated portions of a partially correct response.

2.6 Code Models for Hardware-Specific Adaptation

General code LLMs provide the pretrained capabilities on which RTL-generation systems are commonly built. Qwen2.5-Coder [18], CodeQwen [2], CodeLlama [35], DeepSeek-Coder [11], and CodeGemma [39] represent strong code-generation backbones, but they are not trained to internalize the verification semantics of a particular HDL task. Hardware adaptation methods typically improve these backbones through Verilog corpora, feedback-driven generation, or preference learning. CSCA is orthogonal to the backbone architecture and tokenizer: it supplies a verification-grounded training signal that can be applied after Verilog SFT to localize preference updates at the statement-token level.

3 Motivation and Background

Verilog generation is ultimately judged at the module level: a design is useful only when it compiles and agrees with its specification under verification. This criterion is appropriate for deployment, but it is overly coarse as a learning signal. Generated RTL is often partially correct, and a module-level failure can conceal correct outputs and implementation decisions that should be retained during training. CSCA is motivated by three mismatches between the granularity of simulation feedback and the granularity at which preference optimization assigns credit: *what is verified*, *what is attributed*, and *how strongly the model is updated*.

3.1 From Module-Level Rewards to Signal-Level Supervision

Consider a module with two output signals, s_1 and s_2. A candidate may correctly implement s_1 but use an incorrect condition for s_2. A module-level evaluator assigns this candidate the same failed outcome as a candidate that implements neither signal correctly:

$$R_{\text{mod}}(y) = \mathbb{I}\left[e_y(s_1) = 0 \ \wedge \ e_y(s_2) = 0\right], \tag{1}$$

where $e_y(s)$ is the observed error count of signal s under the testbench. The conjunction is necessary for final functional verification, but it loses information that is valuable for learning. In particular, it cannot distinguish a candidate containing one correct output implementation from a candidate that contains no correct implementation.

This mismatch motivates signal-level supervision. Instead of reducing a module to one binary outcome, CSCA retains a vector of output-specific behavioral scores. It can therefore use the correct implementation of s_1 as training evidence even when s_2 causes the complete module to fail. The central question changes from *"Does this module pass?"* to *"Which observable behavior is supported by this candidate?"* This finer-grained view preserves partially correct RTL as a source of preference information without weakening the module-level criterion used for final evaluation.

3.2 From Static Dependency to Counterfactual Evidence

Signal-level supervision alone does not identify which code fragment deserves credit. A source assignment may be syntactically associated with an output but have no observable effect on the tested behavior. Conversely, conditional control flow, state updates, and shared intermediate signals can make a simple source-level dependency incomplete.

For example, consider two statements `assign mid = a & b;` and `assign out = mid | c;`. Both statements are statically related to `out`. However, static association alone cannot determine whether perturbing either statement changes `out` under the verification traces, whether another assignment or branch overrides the value, or whether `mid` also affects another observable output. Thus, syntactic proximity and AST connectivity do not by themselves establish functional responsibility.

CSCA addresses this attribution mismatch through local counterfactual interventions. It modifies an output-driving statement, recompiles the candidate, and re-executes the same testbench. If a local perturbation increases mismatches for the target signal, the original statement receives stronger evidence of positive functional contribution. The resulting evidence remains conditional on the test distribution and is not a formal proof of causality. Nevertheless, it is more discriminating than static co-occurrence: it tests whether the observed behavior changes when the candidate fragment is changed while the remaining module and external stimuli are held fixed.

3.3 From Binary Masks to Confidence-Weighted Optimization

Even after localizing a signal-related statement, an optimization problem remains: not all selected statements are equally reliable. A binary token mask treats every token in a selected span identically. This assumption is fragile when the statement is only partially correct, when an intervention has little observable effect, or when multiple statements jointly determine the same behavior.

CSCA therefore represents supervision strength continuously. It combines a signal's behavioral agreement with the degradation caused by counterfactual interventions. A statement that is merely associated with an output but whose perturbation does not affect the observed behavior receives little credit. In contrast, a statement that is behaviorally supported and whose perturbation consistently increases target errors receives greater credit. CSCA uses this credit as a statement-token weight in preference optimization: uncertain evidence produces a small update, whereas strongly supported evidence produces a larger update. This design prevents an entire partially correct response from being reinforced uniformly because of a single reliable fragment.

3.4 Verilog Generation and Preference Optimization

Verilog generation requires both syntactic validity and functional agreement with a design specification. Existing benchmarks assess generated modules with compilation and simulation, while prior work has explored instruction tuning, feedback-driven generation, and domain-adapted code models [3, 26–29, 41, 42]. Preference optimization offers an efficient interface for learning from ranked candidate responses [34]. CSCA retains this interface but changes how preferences and update strengths are constructed. A candidate pair is ranked using continuous simulation and counterfactual evidence, and the DPO loss is localized to statement tokens that carry this evidence. Accordingly, CSCA bridges module-level benchmark outcomes and the statement-level decisions that determine generated RTL behavior.

4 Method

4.1 Framework Overview

Figure 1 illustrates the CSCA pipeline. CSCA contains two training stages. In the first stage, we fine-tune a naive code model on prompt–reference-code pairs to obtain an SFT model. In the second stage, the SFT model samples multiple candidate Verilog implementations for each design prompt. CSCA converts behavioral evidence from these candidates into statement-level preference supervision and optimizes the model with weighted DPO.

Formally, let $\mathcal{D} = \{(x, y_{\text{ref}})\}$ denote a set of Verilog design prompts and reference implementations. Starting from a base generator π_0, we first obtain an SFT policy π_{SFT} through full-parameter supervised fine-tuning:

$$\mathcal{L}_{\text{SFT}} = -\mathbb{E}_{(x,y_{\text{ref}})\sim\mathcal{D}}\left[\sum_{t=1}^{|y_{\text{ref}}|}\log \pi_\theta(y_{\text{ref},t} \mid x, y_{\text{ref},<t})\right]. \tag{2}$$

The resulting π_{SFT} provides Verilog-specific generation ability and serves as the initialization for CSCA. For each prompt x, we sample K candidate modules:

$$\mathcal{Y}(x) = \{y_1, y_2, \ldots, y_K\} \sim \pi_{\text{SFT}}(\cdot \mid x). \tag{3}$$

As illustrated in Figure 1(a), CSCA has three subsequent components: (1) continuous behavioral verification ranks candidates according to output-signal agreement with the reference design; (2) counterfactual causal attribution estimates the functional impact of output-driving statements through local interventions; and (3) statement-level weighted DPO converts the resulting credits into localized preference updates. This design preserves useful supervision from partially correct modules rather than discarding every candidate that fails module-level verification.

4.2 Continuous Behavioral Verification

Conventional verification assigns one binary result to an entire module. This is inadequate for generated RTL because a candidate may implement some output signals correctly while failing on others. As shown in Figure 1(b), CSCA executes the reference design and all candidate modules on the same clocked input traces. For each candidate, the testbench observes its output waveforms and compares them with the corresponding reference waveforms signal by signal.

For candidate y_i, let O_i be its observable output signals and let $\mathcal{T} = \{u_1, \ldots, u_N\}$ denote the test vectors. For an output signal $s \in O_i$, we count the number of mismatches:

$$e_i(s) = \sum_{u\in\mathcal{T}} \mathbb{I}\left[f_{y_i}(u, s) \neq f_{y_{\text{ref}}}(u, s)\right], \tag{4}$$

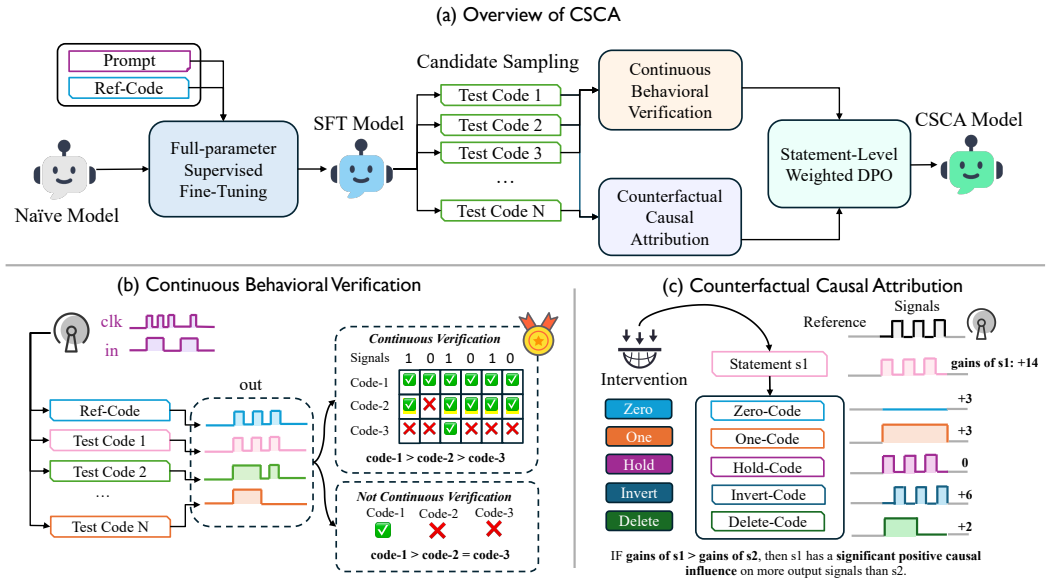

Fig. 1. Overview of CSCA. A naive code model is first fine-tuned on prompt–reference-code pairs to obtain an SFT model. The SFT model samples candidate RTL implementations, which are assessed by continuous behavioral verification and counterfactual causal attribution. The resulting statement-level credits are used to localize weighted DPO updates.

where $f_y(u, s)$ is the value of signal s generated by module y under test vector u. We then define the continuous behavioral correctness of s as

$$r^i_{\text{beh}}(s) = 1 - \frac{e_i(s)}{N}. \tag{5}$$

The vector $\mathbf{r}^i_{\text{beh}} = [r^i_{\text{beh}}(s)]_{s \in O_i}$ captures the partial functional behavior of a candidate. For example, a module may correctly implement one output for all traces while producing incorrect values for another output. CSCA retains this signal-level evidence rather than collapsing the candidate into a single failed label. We aggregate signal scores to obtain a preliminary candidate ranking:

$$R_{\text{beh}}(y_i) = \frac{1}{|O_i|} \sum_{s \in O_i} r^i_{\text{beh}}(s). \tag{6}$$

Compilation failures, elaboration failures, timeouts, and unavailable signal observations are recorded separately and do not receive positive behavioral credit. Because the scores are measured on finite test traces, they represent empirical behavioral evidence rather than formal equivalence guarantees.

4.3 Counterfactual Causal Attribution

Behavioral agreement does not establish whether an RTL statement is responsible for an observed output behavior. CSCA therefore performs local counterfactual interventions. Given a candidate y_i and an output signal s, we locate an eligible assignment statement $a_{i,s}$ that directly drives s. We preserve the source span of $a_{i,s}$ in the generated response for later token-level alignment.

As shown in Figure 1(c), CSCA creates five local counterfactual variants by changing only the selected statement:

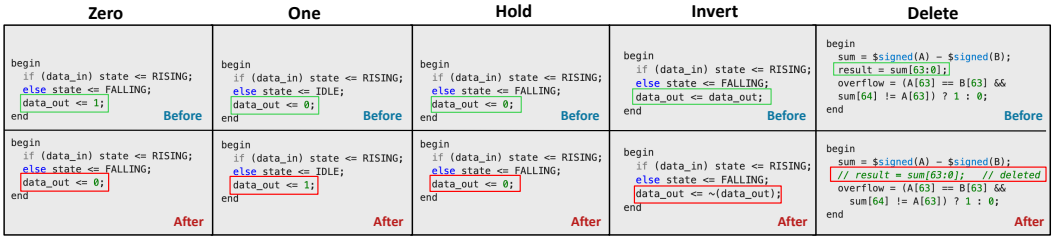

Fig. 2. Examples of CSCA's five local counterfactual interventions. Starting from the same output-driving statement, CSCA creates Zero, One, Hold, Invert, and Delete variants by modifying only the target assignment while preserving the remaining statements in the local `begin...end` block. Each variant is subsequently executed using the original testbench to measure its signal-level behavioral impact.

(1) **Zero**: force the target signal to logic zero, $s \leftarrow$ `'0`;
(2) **One**: force the target signal to logic one, $s \leftarrow$ `'1`;
(3) **Hold**: replace the update with self-assignment, $s \leftarrow s$;
(4) **Invert**: negate the signal value, $s \leftarrow \sim s$;
(5) **Delete**: remove the assignment statement.

Figure 2 illustrates that each transformation is local: it changes only the assignment driving the target signal while retaining the surrounding control context. Let $y_i^{(q,a_{i,s})}$ denote the candidate produced by applying intervention q. All other RTL statements, the module interface, and the testbench remain unchanged. Thus, differences in the output waveforms can be attributed to the local perturbation of $a_{i,s}$.

For a successful intervention q, CSCA measures the change in the mismatch count of the *driven target signal* s. We refer to successfully localized output-driving assignments as eligible statements. An eligible statement has counterfactual support if at least one of its interventions compiles and executes successfully; otherwise, it retains behavioral evidence only.

$$d_i^{(q,a_{i,s})}(s) = \mathrm{clip}\left(\frac{e_i^{(q,a_{i,s})}(s) - e_i(s)}{N}, 0, 1\right), \tag{7}$$

where $e_i^{(q,a_{i,s})}(s)$ is the mismatch count of s after applying intervention q. Here, $\mathrm{clip}(z,0,1) = \min(\max(z,0),1)$ truncates a scalar to $[0,1]$. A positive value means that changing $a_{i,s}$ increases the number of mismatches on the signal it directly drives, providing empirical evidence that the original statement supports the observed behavior of s. Conversely, an intervention that reduces mismatches receives zero counterfactual credit; CSCA does not turn such evidence into a negative token weight.

Let $Q_{i,s}$ be the set of interventions that compile and execute successfully. The target-signal counterfactual contribution is the mean observed degradation:

$$r_{\mathrm{cf}}^{i}(a_{i,s}, s) = \frac{1}{|Q_{i,s}|} \sum_{q \in Q_{i,s}} d_i^{(q,a_{i,s})}(s). \tag{8}$$

This score is deliberately local: it attributes a statement only through the observed behavior of the output signal that the statement directly drives. Accordingly, CSCA does not interpret it as a formal global causal effect over all module outputs. The artifact records the baseline mismatch count, each valid intervention, and the corresponding target-signal mismatch count, which makes every assigned credit traceable to simulation evidence.

We fuse behavioral and counterfactual evidence into a continuous credit:

$$w_i(a_{i,s}, s) = \text{clip}\left(\alpha r^i_{\text{beh}}(s) + (1-\alpha) r^i_{\text{cf}}(a_{i,s}, s), 0, 1\right), \tag{9}$$

where $\alpha \in [0,1]$ balances observed behavioral correctness and counterfactual contribution. If no valid intervention is available, the statement receives no counterfactual evidence, and we use its behavioral score as the fallback credit, i.e., $w_i(a_{i,s}, s) = r^i_{\text{beh}}(s)$. Such statements remain eligible for preference optimization but are distinguished from statements with valid counterfactual evidence.

4.4 Statement-Level Weighted DPO

CSCA constructs preference pairs from candidates sampled for the same prompt. For each candidate, we aggregate its supported statement credits:

$$R_{\text{CSCA}}(y_i) = \frac{1}{|\mathcal{A}_i|} \sum_{a_{i,s} \in \mathcal{A}_i} w_i(a_{i,s}, s), \tag{10}$$

where A_i is the set of eligible output-driving statements successfully localized in y_i. Statements with at least one valid counterfactual intervention use the combined credit in Eq. 9, whereas statements without valid interventions use behavioral correctness as fallback credit. A higher-scoring candidate is selected as the chosen response y^+, and a lower-scoring candidate as the rejected response y^-. Consequently, a globally imperfect candidate can still become a useful preference example when it contains statements with strong behavioral and counterfactual support.

For every supported statement, CSCA maps its response-level character span to token positions. Let ω^+_t and ω^-_t be the token weights of the chosen and rejected responses. Tokens within the span of statement $a_{i,s}$ receive weight $w_i(a_{i,s}, s)$, while tokens outside credited statement spans receive zero weight in the strict setting. Thus, CSCA does not uniformly reinforce an entire preferred module.

For a policy π_θ, frozen reference policy π_{ref}, response y, and token weights $\boldsymbol{\omega}$, we define the weighted log-ratio:

$$\Delta_\theta(x, y, \boldsymbol{\omega}) = \sum_{t=1}^{|y|} \omega_t \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})}. \tag{11}$$

CSCA optimizes the following weighted DPO objective:

$$\mathcal{L}_{\text{CSCA}} = -\mathbb{E}_{(x,y^+,y^-)}\left[\log \sigma\left(\beta\left(\Delta_\theta(x, y^+, \boldsymbol{\omega}^+) - \Delta_\theta(x, y^-, \boldsymbol{\omega}^-)\right)\right)\right], \tag{12}$$

where β controls the preference sharpness. Unlike standard DPO [34], which aggregates all completion tokens uniformly, CSCA concentrates the optimization signal on statements supported by simulation and counterfactual evidence. This localized objective increases the likelihood of functionally informative RTL decisions while avoiding reinforcement of unrelated statements in an otherwise preferred module. Setting all token weights to one recovers standard DPO; assigning binary weights only to relevant spans yields a binary signal-aware variant.

4.5 Experimental Setup

4.5.1 Datasets. Our training dataset is constructed based on CodeV [54], which contains 165K Verilog module-level code samples collected from publicly available GitHub repositories. Following the preprocessing strategy adopted by QiMeng-SALV [53], we remove samples with syntax errors or invalid Verilog implementations, resulting in a high-quality dataset containing 135K valid Verilog modules.

The processed dataset is first utilized to fine-tune the general-purpose code generation model Qwen2.5-Coder-Instruct [18] through supervised fine-tuning (SFT), producing a domain-adapted

Verilog code generator. Given each instruction in the training set, the generator produces five candidate RTL implementations. The original Verilog implementation associated with each instruction is preserved as the reference design and is subsequently used in the signal-aware verification stage to evaluate the signal-level functional equivalence between generated and reference implementations.

4.5.2 Training Configuration. Our training procedure consists of two consecutive stages. In the first stage, we perform supervised fine-tuning (SFT) with full-parameter optimization for two epochs to adapt the pretrained code generation model to the Verilog generation task. In the second stage, we further optimize the model using LoRA-based [16] signal-aware Direct Preference Optimization (DPO) for approximately one epoch (7K optimization steps).

We employ the Adam optimizer [22] with a cosine learning rate decay schedule. The initial learning rate is set to 1.0×10^{-5} for the SFT stage and 5.0×10^{-6} for the signal-aware DPO stage. The global batch size is set to 64, and the maximum sequence length is limited to 2048 tokens. All experiments are conducted under identical training configurations to ensure fair and reproducible comparisons.

4.5.3 Evaluation Metrics. Following previous studies on Verilog code generation [28, 33], we adopt the Pass@k metric as the primary evaluation criterion. Pass@k measures the probability that at least one correct solution is generated among k independent samples for each problem, defined as:

$$\text{Pass@}k = \mathbb{E}_{p\sim\mathcal{P}}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right], \tag{13}$$

where n denotes the total number of generated samples, c represents the number of functionally correct solutions verified through simulation, and $\mathcal{P}$ denotes the distribution of evaluation problems.

Besides functional correctness, we additionally introduce Syntax Pass@k to evaluate the syntactic reliability of generated Verilog programs. Different from Pass@k, Syntax Pass@k considers a generated sample successful when it can be correctly parsed by the Verilog compiler, formulated as:

$$\text{Syntax Pass@}k = \mathbb{E}_{p\sim\mathcal{P}}\left[1 - \frac{\binom{n-s}{k}}{\binom{n}{k}}\right], \tag{14}$$

where s denotes the number of syntactically valid samples among the n generated candidates.

4.5.4 Benchmark. We evaluate our approach on two mainstream RTL-generation benchmarks: **VerilogEval** [33] and **RTLLM** [29]. VerilogEval2.0 [33] comprises two representative subtasks: code completion and specification-to-RTL generation, each containing 156 Verilog design problems. Compared with its predecessor VerilogEval1.0 [27], VerilogEval2.0 broadens task coverage and enhances evaluation fidelity by resolving known limitations in the original benchmark suite. RTLLM v1.1 [29] includes 29 Verilog generation tasks focusing on arithmetic and logic circuits, where each task provides complete module interfaces (port definitions) alongside detailed functional specifications. RTLLM v2.0 extends this benchmark to 50 design instances, further diversifying task types into four categories: arithmetic, control, memory, and miscellaneous logic designs. We perform extensive experiments on VerilogEval2.0 and RTLLM v2.0.

Following prior practice [28], we sweep sampling temperature from 0.0 to 1.0 for hyperparameter analysis. For main-result reporting, we strictly follow the official VerilogEval2.0 protocol [33]: we apply nucleus sampling [14] and report Pass@1 under two decoding configurations: greedy decoding ($T = 0.0$, $n = 1$) and high-temperature stochastic sampling ($T = 0.8$, $n = 5$).

4.5.5 Baselines. As summarized in Table 1, we compare CSCA against a comprehensive suite of baselines. We organize these competitors into three disjoint categories:

- **General-purpose foundation models**: This group consists of Llama3.1 [10], Qwen2.5 [49], Qwen3 [48], DeepSeek-V3 [6], GPT-5.4-mini [37], and Gemini-3.1-pro [40]. These large models exhibit strong cross-domain reasoning capacity. Notably, DeepSeek-V3 (671B) delivers competitive RTL generation performance without hardware-specific fine-tuning, though its enormous parameter footprint imposes substantial overhead for real-world deployment.
- **General coding models**: We include CodeQwen1.5 [2], Qwen2.5-Coder-Instruct [18], CodeLlama [35], DeepSeek-Coder [11], and CodeGemma [39]. Pre-trained on large-scale software code corpora, they achieve solid performance on conventional programming tasks. Even so, they lack specialized inductive biases for hardware description languages, leading to clear performance degradation on sophisticated Verilog synthesis tasks, even for variants with up to 70B parameters.
- **Verilog-specific generation models**: We benchmark against a set of state-of-the-art specific approaches for RTL code generation, including RTLCoder [28], CodeV [54], Origen [5], VeriSeek [44], VeriPrefer [43], and QiMeng-SALV [53]. All of them improve Verilog capability via domain-specific adaptation strategies such as curated Verilog corpus construction, simulation-based reinforcement learning, or hardware-aware preference optimization:
  - **CodeV [54]**: constructs open-source Verilog corpora and applies instruction tuning to repurpose LLMs for hardware generation.
  - **RTLCoder [28]**: adopts reinforcement learning for RTL generation, using simulation-derived module-level functional correctness as reward.
  - **Origen [5]**: adapts LLMs for hardware design with Verilog-focused instruction tuning and hardware-aware training objectives.
  - **VeriSeek [44]**: integrates domain-specialized training and verification-guided optimization to boost functional correctness of generated RTL.
  - **VeriPrefer [43]**: performs Verilog preference optimization; preference pairs are derived from hardware simulation results to align model outputs with target circuit behaviors.
  - **QiMeng-SALV [53]**: augments reinforcement learning with fine-grained signal-level simulation feedback, overcoming the limitations of coarse module-level reward signals.

5 Evaluation Results

We evaluate CSCA as a training-signal design rather than as a new backbone model. Our goal is to establish whether the additional signal-level evidence improves RTL generation, to separate the roles of behavioral and counterfactual evidence, and to quantify the cost of obtaining that evidence. Unless otherwise stated, all CSCA variants use the same Qwen2.5-Coder-7B-Instruct backbone, CodeV-derived training corpus, rollout budget, DPO hyperparameters, prompts, and simulation harness. We report functional Pass@1 and Pass@5; RTLLM additionally reports syntax Pass@1 and Pass@5. Pass@1 uses one generated candidate, while Pass@5 is estimated from five independently sampled candidates. Higher values are better throughout.

5.1 RQ1: How much does CSCA improve functional RTL generation compared with the foundation model, Verilog-specific models, and state-of-the-art methods?

Table 1 summarizes the performance of various large models on Verilog code-generation benchmarks, including foundation general-purpose models, general coding models, and Verilog-specific models. We evaluate models on two benchmarks: VerilogEval 2.0 (code completion and specification-to-RTL) and RTLLM v2.0, under both greedy decoding ($T = 0$) and stochastic sampling ($T = 0.8$). Pass@1 measures the success rate of a single generated sample, while Pass@5 reflects the probability of producing at least one correct design among five samples.

First, from the perspective of model scale, hardware generation performance generally improves with model size for general foundation models. Gemini-3.1-pro achieves the highest among all

Table 1. Main Results on VerilogEval2.0 and RTLLM v2.0 Benchmarks (%). $T = 0$ denotes greedy decoding, for which only Pass@1 can be evaluated since a single sample is generated. For $T = 0.8$, we report both Pass@1 and Pass@5. Bold values indicate the best performance among Verilog-specific models.

Type	Model	Size	VerilogEval (Completion)			VerilogEval (Specification)			RTLLM v2.0		
			$T = 0$	$T = 0.8$		$T = 0$	$T = 0.8$		$T = 0$	$T = 0.8$	
			Pass@1	Pass@1	Pass@5	Pass@1	Pass@1	Pass@5	Pass@1	Pass@1	Pass@5
Foundation General Model	Llama3.1	8B	2.6	2.1	6.4	19.1	16.8	32.1	6.0	6.0	16.0
	Qwen2.5-7B	7B	21.8	12.8	35.3	24.4	21.4	42.3	24.0	19.6	42.0
	Qwen3-30B-A3B	30B	43.3	43.5	51.9	44.2	46.2	57.1	38.0	39.6	52.0
	DeepSeek-v3	671B	68.0	66.1	71.8	68.8	66.9	72.4	56.0	59.1	71.5
	GPT-5.4-mini	–	52.6	53.3	69.9	50.0	49.4	62.2	58.0	58.0	66.0
	Gemini-3.1-pro	–	75.0	73.1	80.8	78.9	76.3	83.3	62.0	63.2	74.0
General Coding Model	CodeQwen1.5	7B	22.5	16.2	35.9	1.9	15.0	34.6	28.0	25.8	29.0
	Qwen2.5-Coder	7B	32.7	27.9	43.0	23.0	22.0	50.6	36.0	33.2	52.5
	CodeLlama	70B	37.2	29.0	43.6	34.9	25.3	51.9	36.0	32.8	50.0
	DeepSeek Coder	33B	35.3	29.3	44.2	21.7	19.5	34.6	34.0	31.2	50.0
	CodeGemma	7B	8.3	8.7	15.4	6.6	9.5	14.1	6.0	4.8	16.0
Verilog-Specific Model	RTLCoder	6.7B	35.9	27.6	39.7	36.8	30.9	51.9	44.0	43.5	48.1
	CodeV	7B	52.6	45.0	60.3	44.8	37.4	52.6	42.0	41.0	54.0
	Origen	7B	53.8	47.9	64.1	48.7	46.8	56.4	40.0	37.2	52.0
	VeriSeek	6.7B	47.4	43.4	57.7	28.8	15.9	36.5	36.0	31.9	54.2
	VeriPrefer	7B	49.7	48.7	64.1	44.9	40.8	55.1	44.0	39.6	48.0
	QiMeng-SALV	7B	54.5	49.1	64.7	46.2	43.7	59.6	46.0	41.2	58.0
	CSCA(Ours)	7B	**56.4**	**55.1**	**66.0**	**49.3**	**47.4**	**62.2**	**50.0**	**47.2**	**60.0**

Table 2. Credit component ablation study of CSCA ($T = 0.8$). All variants share the same base model, prompt set, and simulation budget. VerilogEval 2.0 is split into Code Completion and Specification-to-RTL subtasks. The largest improvement among all ablations for each metric is marked in bold.

Configuration	VerilogEval 2.0				RTLLM v2.0			
	Code Completion		Spec.-to-RTL		Syntax Pass@1	Syntax Pass@5	Pass@1	Pass@5
	Pass@1	Pass@5	Pass@1	Pass@5				
Base Model (Qwen2.5-Coder)	27.9	43.0	22.0	50.6	74.0	88.0	33.2	52.5
+ SFT	32.7	44.2	31.0	53.8	75.2	96.2	35.5	50.0
+ SFT + Counterfactual Effect	49.7	65.6	43.2	62.0	84.5	94.0	41.0	55.3
+ SFT + Behavioral Correctness	49.3	65.2	42.8	61.3	84.8	94.0	41.2	56.0
Full CSCA	55.1	66.0	47.4	62.2	92.0	98.1	47.2	60.0

general foundation models. DeepSeek-v3 (671B) achieves 68.0% Pass@1 on VerilogEval completion under greedy decoding. Nevertheless, CSCA with 7B parameters achieves comparable results to this huge general model on most metrics. CSCA reaches 56.4% Pass@1 ($T = 0$) and 66.0% Pass@5 ($T = 0.8$) on VerilogEval completion, consistently outperforming larger general coding baselines such as 33B DeepSeek Coder and 70B CodeLlama. These results highlight the effectiveness of domain-specific optimization even with a relatively small backbone. Even a 7B model can surpass models with tens of billions of parameters when equipped with tailored hardware reasoning mechanisms.

Second, regarding training and modeling strategies, models with different training paradigms exhibit clear performance gaps. General foundation models and general code models lack dedicated Verilog inductive biases. Existing Verilog-specific baselines, including RTLCoder, CodeV, Origen, and QiMeng-SALV, adopt conventional domain fine-tuning. The strongest baseline, QiMeng-SALV,

obtains 54.5% Pass@1 and 64.7% Pass@5 on VerilogEval completion. In contrast, our CSCA adopts counterfactual structural intervention learning, pushing the performance to 56.4% Pass@1 ($T = 0$) and 66.0% Pass@5 ($T = 0.8$). CSCA also achieves 47.4% Pass@1 ($T = 0.8$) and 62.2% Pass@5 ($T = 0.8$) on VerilogEval specification-to-RTL, as well as 50.0% Pass@1 on RTLLM v2.0 under greedy decoding. These consistent improvements under both greedy ($T = 0$) and stochastic sampling ($T = 0.8$) demonstrate consistent improvements over the evaluated domain fine-tuning and verification-guided baselines.

Third, in terms of cross-dataset generalization, CSCA maintains stable and consistent gains across all three challenging benchmarks: VerilogEval code completion, VerilogEval specification-to-RTL, and RTLLM v2.0. On VerilogEval completion, CSCA outperforms all Verilog-specialized baselines. On the more challenging specification-to-RTL subtask, CSCA attains 49.3% Pass@1 ($T = 0$) and 62.2% Pass@5 ($T = 0.8$). On RTLLM v2.0, CSCA achieves 50.0% Pass@1 ($T = 0$) and 60.0% Pass@5 ($T = 0.8$), exceeding QiMeng-SALV (46.0% Pass@1, 58.0% Pass@5). Unlike several baselines that achieve strong results on one dataset but suffer obvious performance drops on others, CSCA retains competitive performance across different task formats, demonstrating consistent effectiveness across different RTL-generation task formats.

5.2 RQ2: How do behavioral correctness and counterfactual intervention effects contribute to the performance of CSCA?

To investigate the individual and combined contributions of behavioral correctness modeling and counterfactual intervention, we conduct a controlled ablation study by disabling each component while keeping the backbone model, training prompts, and computational budget identical. As shown in Table 2, both behavioral-only and counterfactual-only variants consistently outperform the vanilla SFT baseline, demonstrating that each component serves as an effective local supervision signal for Verilog generation.

Nevertheless, neither single-component variant is sufficient to reproduce the full performance of CSCA. On the challenging VerilogEval specification-to-RTL task, the counterfactual-only and behavioral-only variants achieve only 43.2% and 42.8% Pass@1, respectively, while the full CSCA framework yields a substantially higher score of 47.4%. Consistent trends are observed on the RTLLM v2.0 benchmark: individual counterfactual and behavioral branches obtain merely 41.0% and 41.2% functional Pass@1, whereas full CSCA improves this metric to 47.2%.

These results indicate the complementary nature of the two designs. Behavioral correctness learning enhances the model's ability to generate functionally consistent code by leveraging empirically valid execution traces. In contrast, counterfactual intervention further refines structural reasoning by perturbing key circuit assignments and verifying causal behavior changes. The integration of empirical behavioral constraints and causal counterfactual supervision jointly eliminates spurious patterns and yields superior functional accuracy as well as higher syntax pass rates. Therefore, the performance gains of CSCA stem not from a single supervisory signal, but from the synergistic combination of behavioral verification and causal intervention learning.

5.3 RQ3: How sensitive is CSCA to hyperparameters including sampling temperature and the behavioral-counterfactual trade-off coefficient?

We investigate the hyperparameter sensitivity of CSCA from two critical dimensions, the **sampling temperature** T that controls generation diversity and the **trade-off coefficient** α that balances behavioral correctness and counterfactual causal evidence.

First, we analyze the effect of decoding temperature T across a wide range from 0.0 (greedy decoding) to 1.0, as summarized in Table 3. Greedy decoding ($T = 0.0$) produces deterministic outputs and achieves the strongest Pass@1 performance on VerilogEval Code Completion (56.4%), yet it

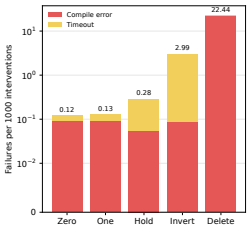

Fig. 3. Simulation failure rates of five counterfactual interventions in the dataset, counting compile errors and timeouts per 1000 interventions.

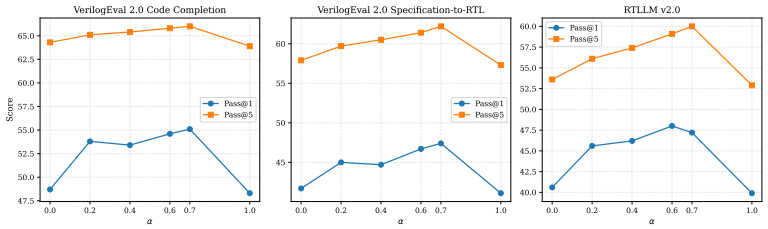

Fig. 4. Sensitivity analysis of behavioral weight α. Three subplots correspond to VerilogEval 2.0 Code Completion, VerilogEval 2.0 Spec.-to-RTL, and RTLLM v2.0, respectively. Minor local fluctuations can be observed, e.g., some Pass@1 metrics at $\alpha = 0.2$ slightly outperform $\alpha = 0.4$. Overall performance peaks at the default setting $\alpha = 0.7$, and degrades when relying solely on a single reward signal ($\alpha = 0.0$ or $\alpha = 1.0$).

cannot benefit from diverse sampling and thus lacks valid Pass@5 evaluation. As T increases within a moderate range ($0.1 \sim 0.8$), the increased generation diversity consistently improves Pass@5 scores across all benchmarks. Specifically, when $T = 0.8$, CSCA achieves the optimal comprehensive performance, yielding 66.0% Pass@5 for code completion, 62.2% Pass@5 for specification-to-RTL, and 60.0% Pass@5 on RTLLM v2.0, alongside competitive Pass@1 results. However, excessively high temperature ($T \geq 0.9$) introduces unreasonable circuit perturbations and noisy logic generation, leading to consistent performance drops in both functional and syntax metrics. These results indicate that a moderate temperature strikes the best balance between deterministic correctness and generative diversity for hardware circuit generation.

Second, we evaluate the sensitivity of the behavioral-counterfactual trade-off coefficient α in Eq. 9, where $\alpha = 0$ activates only counterfactual intervention supervision and $\alpha = 1$ relies purely on behavioral correctness signals. As illustrated in Figure 4, models trained with single supervision sources ($\alpha = 0$ or $\alpha = 1$) yield the lowest overall performance. Pure counterfactual training ($\alpha = 0$) obtains only 48.7%/64.3%, 41.7%/57.9%, and 40.6%/53.6% (Pass@1/Pass@5) on the three benchmarks, while pure behavioral training ($\alpha = 1$) achieves 48.3%/63.9%, 41.1%/57.3%, and 39.9%/52.9%, respectively. In contrast, the hybrid setting with $\alpha = 0.7$ consistently delivers the best results across all tasks: 55.1%/66.0% for VerilogEval completion, 47.4%/62.2% for specification-to-RTL, and 47.2%/60.0% for RTLLM v2.0. Notably, the performance curves are non-monotonic with minor local fluctuations, which is reasonable given randomized sampling during simulation. Neither purely behavioral nor purely counterfactual supervision is optimal, and intermediate hybrid weighting achieves superior performance.

5.4 RQ4: How reliable are the five types of RTL-level counterfactual interventions under simulation?

Before leveraging counterfactual outcomes as supervision signals, it is critical to quantify simulation failure rates across different intervention operations. Excessive compile errors or timeouts would render the counterfactual signal noisy and reduce the amount of usable training data. We therefore evaluate execution reliability for Zero, One, Hold, Invert, and Delete interventions, as reported in Figure 3. The y-axis denotes non-successful simulation cases per one-thousand intervention instances, split into compile error (red) and timeout (yellow).

Table 3. Functional and syntax pass rates (%) over temperature T. $T = 0$ denotes greedy decoding ($N = 1$), where Pass@5 metrics are not applicable. For $T \geq 0$, $N = 5$ samples are used.

Temperature	VerilogEval 2.0				RTLLM v2.0			
	Code Completion		Spec.-to-RTL		Syntax Pass@1	Syntax Pass@5	Pass@1	Pass@5
	Pass@1	Pass@5	Pass@1	Pass@5				
$T = 0.0^{\dagger}$	56.4	—	49.3	—	88.0	—	50.0	—
$T = 0.1$	53.7	57.7	46.3	52.6	87.2	90.0	45.2	54.0
$T = 0.2$	54.1	59.6	45.1	51.9	87.2	92.0	45.6	56.0
$T = 0.3$	55.1	60.9	45.4	56.4	86.8	94.0	47.2	60.0
$T = 0.4$	54.5	63.5	46.0	59.0	85.2	92.0	45.6	58.0
$T = 0.5$	53.3	63.5	44.6	59.0	85.6	90.0	46.0	58.0
$T = 0.6$	52.1	62.8	43.7	59.6	86.8	94.0	45.6	58.0
$T = 0.7$	48.7	64.1	44.4	61.5	86.0	94.0	42.4	54.0
$T = 0.8$	55.1	66.0	47.4	62.2	92.0	98.1	47.2	60.0
$T = 0.9$	47.9	66.0	41.5	62.2	83.2	94.0	41.2	60.0
$T = 1.0$	45.0	60.3	41.2	60.9	82.4	92.0	41.6	56.0

Zero and One interventions exhibit the highest robustness, with overall failure rates of only 0.12 and 0.13 per 1000 interventions, respectively, where failures are almost negligible. The Hold operation also maintains high reliability with 0.28 failures per 1000. By contrast, the Invert intervention yields a notably higher failure rate of 2.99 per 1000, and most failures originate from simulation timeouts induced by altered control-flow logic. The Delete intervention suffers the worst reliability at 22.44 failures per 1000 interventions; nearly all failures stem from compilation errors triggered by removing driving assignments for circuit signals.

In aggregate, four out of five intervention types achieve success rates above 99.7%. Even for Delete, the least-stable primitive, the success rate still reaches 97.76%. Such statistics confirm that our counterfactual intervention pipeline can operate at scale. During training, we simply discard samples with failed simulation rather than treating them as positive or negative causal evidence, mitigating the adverse impact of intervention failures.

5.5 RQ5: What is the distribution and informativeness of counterfactual gains obtained from RTL counterfactual interventions?

To verify whether our RTL counterfactual perturbations can yield useful causal supervision signals instead of mostly trivial behavior-preserving modifications, we analyze the counterfactual gain collected from RTL-level counterfactual interventions using 893,225 signal-level samples, as illustrated in Figure 5. (1) The donut chart in Figure 5(a) categorizes all samples by the outcome of counterfactual intervention. Among all records, 53.2% of perturbations induce no behavioral change of the circuit after assignment modification. The remaining 46.8% produce nonzero counterfactual gains, split into 36.1% positive gain and 10.7% negative gain. Positive gain means intervention changes circuit behavior following the expected causal direction, while negative gain represents counterintuitive behavioral shifts. (2) Figure 5(b) illustrates the magnitude of nonzero counterfactual gains, covering 417,631 records with an overall mean gain of 0.157. The histogram shows that positive gains dominate the informative samples, with many records reaching high gain values close to 1.0.

These observations demonstrate that the counterfactual interventions are capable of generating meaningful causal supervision signals for a substantial subset of signal assignments, rather than

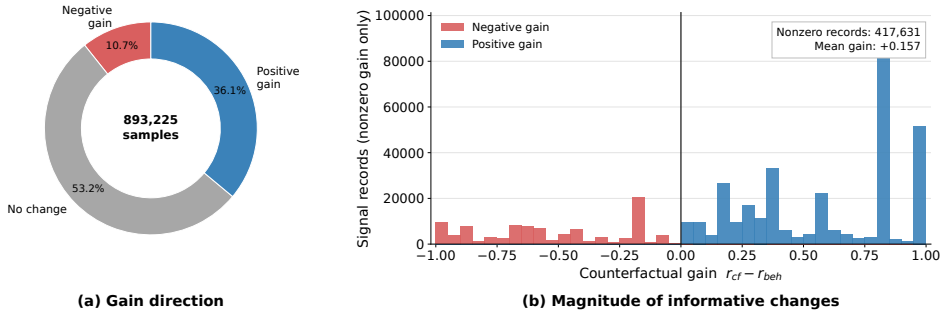

Fig. 5. Counterfactual gain statistics from 893 225 RTL intervention samples. (a) Distribution of intervention outcomes. (b) Histogram of nonzero counterfactual gains for informative records.

merely producing trivial, behavior-invariant perturbations. The existence of both positive and negative gains also reflects the complex causal dependency within RTL circuits.

5.6 RQ6: What are the training and inference computational costs incurred by CSCA?

We distinguish one-time SFT adaptation, the additional CSCA preference-optimization stage, and evaluation. The full-parameter SFT run trained a roughly 7.6B-parameter model on 205,728 examples for two epochs using eight GPUs. Its logged training time was 20 h 18 min, corresponding to 162.4 GPU-hours. Starting from this SFT checkpoint, strict signal-token CSCA-DPO trained 20.19M LoRA parameters (0.264% of the 7.64B total parameters) on 86,534 preference examples for one epoch. This stage completed in 3 hours and 33 min on four GPUs, or 14.2 GPU-hours. Thus, CSCA preference optimization adds approximately 8.8% to the observed GPU budget of the preceding SFT run, without requiring a second full-parameter adaptation stage. For five-sample inference on one GPU, VerilogEval specification-to-RTL generated 780 candidates for 156 tasks in 32 seconds after initialization (about 55 seconds end-to-end); VerilogEval code completion generated the same number of candidates in 30 seconds (about 51 seconds end-to-end). RTLLM generated 250 candidates for 50 tasks in 27 seconds, with an end-to-end logged runtime of about 50 seconds. These measurements are reported as observed costs rather than a normalized throughput comparison, since the SFT and DPO runs use different GPU counts, data volumes, epochs, and batch configurations.

5.7 Qualitative Case Study

To complement the aggregate quantitative results, we examine paired RTL candidates generated by the SFT baseline and our CSCA method on the VerilogEval specification-to-RTL task. Samples are collected under identical decoding hyperparameters ($T = 0.8$, $p = 0.95$). We filter case candidates with strict criteria: both generated RTLs compile successfully, the CSCA candidate passes full functional simulation, whereas the SFT candidate fails simulation. This filter excludes purely syntactic fixes and isolates semantic RTL bugs responsible for functional discrepancies.

Table 4 shows two representative examples. For `Prob030_popcount255`, the SFT candidate incorrectly sums 8-bit input slices as integer values instead of counting individual bits, yielding a 0.0% pass rate across all 215 testbench samples. The CSCA implementation uses a combinational loop to accumulate each input bit, achieving 100.0% test coverage. For `Prob037_review2015_count1k`, the SFT baseline contains two sequential-design bugs: asynchronous reset and an unbounded counter, violating the specification's requirement for synchronous reset and modulo-1000 wrapping.

Table 4. Representative functional improvements over the SFT baseline. Both candidates compile. The benchmark testbench reports passing samples and pass rate.

Task	Verilog generated by SFT	Verilog generated by CSCA

`Prob030_popcount255`	`assign count_8bit = in[7:0] + in[15:8] + ··· + in[255:248];`
	Adds byte values rather than their constituent bits.
		`always @(*) begin`
		`  pop_count = 0;`
		`  for (integer i=0; i<255; i=i+1)`
		`  pop_count = pop_count + in[i];`
		`end`

Testbench	0/215	215/215
Pass rate	0.0%	100.0%

`Prob037_review2015_count1k`	`always @(posedge clk or posedge reset)`
	`  if (reset) counter <= 0;`
	`  else counter <= counter + 1;`
	Uses asynchronous reset and never wraps at 1000.
		`always @(posedge clk) begin`
		`  if (reset) q <= 0;`
		`  else if (q == 10'd999) q <= 0;`
		`  else q <= q + 1;`
		`end`

Testbench	4896/8027	8027/8027
Pass rate	61.0%	100.0%

This yields a 61.0% pass rate (4896/8027). CSCA explicitly enforces synchronous reset and the 1000-cycle wrap condition, passing all 8027 test cases with a 100.0% pass rate.

Together, these examples illustrate that CSCA improves not merely syntax parsability, but core RTL semantics for both combinational bitwise arithmetic and sequential control logic.

6 Threats to Validity and Limitations

Counterfactual interventions only yield empirical evidence under finite testbench traces and cannot guarantee formal circuit equivalence across all inputs; to mitigate this risk, we strictly follow standardized benchmark testbenches and clarify that our credit reflects test-dependent empirical behavior instead of global formal causality. Source-level perturbation and attribution are error-prone for shared assignments, macros, generated blocks, and complex sequential state, which we alleviate by limiting interventions to combinational output-driving assignments and skipping credit computation for unsupported constructs. Experiments built on a single base model and one training corpus pose threats to external validity, and we reduce this concern via evaluation across multiple diverse RTL benchmarks and comprehensive baseline comparisons, while noting further cross-model and cross-dataset studies are needed for broader conclusions. Finally, offline DPO using static pre-built preference pairs may underutilize novel candidates generated during training; we mitigate this by adopting an offline setting for high reproducibility and open-sourcing our full artifact.

7 Reproducibility

Our artifact is publicly available at https://anonymous.4open.science/r/CSCA-3B8C. It includes data-construction scripts, reference training and environment configurations, exact decoding hyperparameters, and benchmark evaluation commands. Following the step-by-step instructions provided in the repository, one can reproduce our experiments.

8 Conclusion

This paper presented Causal Signal Credit Assignment (CSCA), a counterfactual preference-optimization framework for Verilog generation. CSCA converts partially correct RTL modules into localized supervision by combining continuous signal-level behavioral verification with local counterfactual interventions on output-driving statements. The resulting statement credits are mapped

to token weights for strict weighted DPO, allowing the model to favor functionally supported RTL decisions without uniformly reinforcing an entire partially correct response. Experiments across widely-used benchmarks VerilogEval 2.0 and RTLLM v2.0 show that integrating behavioral and counterfactual evidence improves over using either source alone.

Future work can extend CSCA to richer sequential and inter-module dependencies, broaden the set of semantics-preserving local interventions, and combine counterfactual simulation with formal-verification-backed evidence. These extensions can further enrich statement-level credit assignment for complex hardware-design tasks.

References

[1] Ahmed Allam and Mohamed Shalan. 2024. RTL-Repo: A Benchmark for Evaluating LLMs on Large-Scale RTL Design Projects. In *2024 IEEE LLM Aided Design Workshop (LAD)*. doi:10.1109/LAD62341.2024.10691810

[2] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiao Deng, Yige Fan, Wenbin Ge, Yizhu Han, Fei Huang, Binyuan Hui, Lei Ji, Menglin Li, Junyang Lin, Ruisong Lin, Dayi Liu, Guoyang Liu, Chengyu Lu, Kai Lu, Jun Ma, Rui Men, Xuancheng Ren, Xinran Ren, Chengqi Tan, Songlin Tan, Jianhong Tu, Peng Wang, Shuo Wang, Wenhui Wang, Siyuan Wu, Bo Xu, Jingning Xu, An Yang, Hao Yang, Jian Yang, Song Yang, Yuan Yao, Bo Yu, Hongcheng Yuan, Zihan Yuan, Jian Zhang, Xin Zhang, Yongliang Zhang, Zhaoxiang Zhang, Chang Zhou, Jingren Zhou, Xiaohui Zhou, and Tao Zhu. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609* (2023).

[3] Jason Blocklove, Siddharth Garg, Ramesh Karri, and Hammond Pearce. 2023. Chip-Chat: Challenges and Opportunities in Conversational Hardware Design. In *2023 ACM/IEEE Workshop on Machine Learning for CAD*. 1–6. doi:10.1109/MLCAD58807.2023.10299874

[4] Jialun Cao, Zhiyong Chen, Jiarong Wu, Shing-Chi Cheung, and Chang Xu. 2024. JavaBench: A Benchmark of Object-Oriented Code Generation for Evaluating Large Language Models. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 870–882.

[5] Fan Cui, Chenyang Yin, Kexing Zhou, Youwei Xiao, Guangyu Sun, Qiang Xu, Qipeng Guo, Yun Liang, Xingcheng Zhang, Demin Song, et al. 2024. Origen: Enhancing rtl code generation with code-to-code augmentation and self-reflection. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.

[6] DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437* (2024).

[7] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. 2023. Automated Repair of Programs from Large Language Models. In *Proceedings of the 45th International Conference on Software Engineering*. 1469–1481.

[8] Daniel D. Gajski, Nikil Dutt, Allen C-H Wu, and Steve Y-L Lin. 1992. *High-Level Synthesis: Introduction to Chip and System Design*. Springer.

[9] Xinyu Gao, Yun Xiong, Deze Wang, Zhenhan Guan, Zejian Shi, Haofen Wang, and Shanshan Li. 2024. Preference-Guided Refactored Tuning for Retrieval Augmented Code Generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 65–77.

[10] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024). https://arxiv.org/abs/2407.21783

[11] Daya Guo, Qihao Zhu, Dejian Yang, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024). doi:10.48550/arXiv.2401.14196

[12] Lianghong Guo, Yanlin Wang, Ensheng Shi, Wanjun Zhong, Hongyu Zhang, Jiachi Chen, Ruikai Zhang, Yuchi Ma, and Zibin Zheng. 2024. When to Stop? Towards Efficient Code Generation in Large Language Models with Excess Token Prevention. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1073–1085.

[13] Yifeng He, Jiabo Huang, Yuyang Rong, Yiwen Guo, Ethan Wang, and Hao Chen. 2024. UniTSyn: A Large-Scale Dataset Capable of Enhancing the Prowess of Large Language Models for Program Testing. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1061–1072.

[14] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2020. The Curious Case of Neural Text Degeneration. In *International Conference on Learning Representations (ICLR)*.

[15] Soneya Binta Hossain, Nan Jiang, Qiang Zhou, Xiaopeng Li, Wen-Hao Chiang, Yingjun Lyu, Hoan Nguyen, and Omer Tripp. 2024. A deep dive into large language models for automated bug localization and repair. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1471–1493.

[16] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*.

[17] Junjie Huang, Daya Guo, Chenglong Wang, Jiazhen Gu, Shuai Lu, Jeevana Priya Inala, Cong Yan, Jianfeng Gao, Nan Duan, and Michael R. Lyu. 2024. Contextualized Data-Wrangling Code Generation in Computational Notebooks. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*.

[18] Binyuan Hui, Jian Yang, Zeyu Cui, et al. 2024. Qwen2.5-Coder Technical Report. *arXiv preprint arXiv:2409.12186* (2024).

[19] Yue Jia and Mark Harman. 2011. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering* 37, 5 (2011), 649–678. doi:10.1109/TSE.2010.62

[20] Zongze Jiang, Ming Wen, Jialun Cao, Xuanhua Shi, and Hai Jin. 2024. Towards Understanding the Effectiveness of Large Language Models on Directed Test Input Generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*.

[21] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large Language Models are Few-shot Testers: Exploring LLM-based General Bug Reproduction. In *Proceedings of the 45th International Conference on Software Engineering*. 2312–2323.

[22] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR)*.

[23] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMOSA: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *Proceedings of the 45th International Conference on Software Engineering*. 919–931. doi:10.1109/ICSE48619.2023.00085

[24] Jia Li, Yongmin Li, Ge Li, Zhi Jin, Yiyang Hao, and Xing Hu. 2023. SkCoder: A Sketch-based Approach for Automatic Code Generation. In *Proceedings of the 45th International Conference on Software Engineering*. 2124–2135.

[25] Zhong Li, Chong Zhang, Minxue Pan, Tian Zhang, and Xuandong Li. 2024. AACEGEN: Attention Guided Adversarial Code Example Generation for Deep Code Models. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1245–1257.

[26] Mingjie Liu, Teodor-Dumitru Ene, Robert Kirby, et al. 2023. ChipNeMo: Domain-Adapted LLMs for Chip Design. *arXiv preprint arXiv:2311.00176* (2023). doi:10.48550/arXiv.2311.00176

[27] Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating Large Language Models for Verilog Code Generation. In *Proceedings of the 2023 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 1–8. doi:10.1109/ICCAD57390.2023.10323812

[28] Shang Liu, Wenji Fang, Yao Lu, et al. 2025. RTLCoder: Fully Open-Source and Efficient LLM-Assisted RTL Code Generation Technique. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 4 (2025), 1448–1461. doi:10.1109/TCAD.2024.3483089

[29] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2024. RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model. In *2024 Asia and South Pacific Design Automation Conference*. 722–727. doi:10.1109/ASP-DAC58780.2024.10473904

[30] Grant Martin and Gary Smith. 2009. High-level synthesis: past, present, and future. *IEEE Design & Test of Computers* 26, 4 (2009), 18–25.

[31] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binquan Zhang, ChenXue Wang, Shichao Liu, and Qing Wang. 2024. Clarifygpt: A framework for enhancing llm-based code generation via requirements clarification. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 2332–2354.

[32] Judea Pearl. 2009. *Causality: Models, Reasoning, and Inference* (2 ed.). Cambridge University Press.

[33] Nathaniel Pinckney, Christopher Batten, Mingjie Liu, Haoxing Ren, and Brucek Khailany. 2024. Revisiting VerilogEval: Newer LLMs, In-Context Learning, and Specification-to-RTL Tasks. *arXiv preprint arXiv:2408.11053* (2024).

[34] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Advances in Neural Information Processing Systems*, Vol. 36. 53728–53741.

[35] Baptiste Rozière, Jan E. Lacroix, Lowik Chanussot Jallot, et al. 2023. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950* (2023). https://arxiv.org/abs/2308.12950

[36] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347* (2017). doi:10.48550/arXiv.1707.06347

[37] Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. 2025. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267* (2025).

[38] Zhihong Sun, Yao Wan, Jia Li, Hongyu Zhang, Zhi Jin, Ge Li, and Chen Lyu. 2024. Sifting through the Chaff: On Utilizing Execution Feedback for Ranking the Generated Code Candidates. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*.

[39] CodeGemma Team, Heri Zhao, Jeffrey Hui, Joshua Howland, Nam Nguyen, Siqi Zuo, Andrea Hu, Christopher A Choquette-Choo, Jingyue Shen, Joe Kelley, et al. 2024. Codegemma: Open code models based on gemma. *arXiv preprint arXiv:2406.11409* (2024).

[40] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv*

preprint arXiv:2312.11805 (2023).

[41] Shailja Thakur, Baleegh Ahmad, Zhenxing Fan, Hammond Pearce, Benjamin Tan, Ramesh Karri, Brendan Dolan-Gavitt, and Siddharth Garg. 2023. Benchmarking Large Language Models for Automated Verilog RTL Code Generation. In *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1–6. doi:10.23919/DATE56975.2023.10137086

[42] Shailja Thakur, Jason Blocklove, Hammond Pearce, Benjamin Tan, Siddharth Garg, and Ramesh Karri. 2023. AutoChip: Automating HDL Generation Using LLM Feedback. *arXiv preprint arXiv:2311.04887* (2023). doi:10.48550/arXiv.2311.04887

[43] Ning Wang, Bingkun Yao, Jie Zhou, Yuchen Hu, Xi Wang, Nan Guan, and Zhe Jiang. 2025. Insights from verification: Training a verilog generation LLM with reinforcement learning with testbench feedback. *arXiv preprint arXiv:2504.15804* (2025).

[44] Ning Wang, Bingkun Yao, Jie Zhou, Xi Wang, Zhe Jiang, and Nan Guan. 2024. Large language model for verilog generation with golden code feedback. *arXiv preprint arXiv:2407.18271* (2024).

[45] Zejun Wang, Kaibo Liu, Ge Li, and Zhi Jin. 2024. SlicePromptTest4J: High-coverage Test Generation using Large Language Models via Method Slicing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*.

[46] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal Fuzzing with Large Language Models. In *Proceedings of the 46th International Conference on Software Engineering*. 126:1–126:13. doi:10.1145/3597503.3639121

[47] Zhiyi Xue, Liangguo Li, Senyue Tian, Xiaohong Chen, Pingping Li, Liangyu Chen, Tingting Jiang, and Min Zhang. 2024. LLM4Fin: Fully Automating LLM-Powered Test Case Generation for FinTech Software Acceptance Testing. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*.

[48] An Yang, Anfeng Li, Baosong Yang, et al. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025). https://arxiv.org/abs/2505.09388

[49] An Yang, Baosong Yang, Beichen Zhang, et al. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024). doi:10.48550/arXiv.2412.15115

[50] Aidan Z. H. Yang, Claire Le Goues, Ruben Martins, and Vincent J. Hellendoorn. 2024. Large Language Models for Test-Free Fault Localization. In *Proceedings of the 46th International Conference on Software Engineering*. 17:1–17:12.

[51] Huan Zhang, Wei Cheng, Yuhan Wu, and Wei Hu. 2024. A Pair Programming Framework for Code Generation via Multi-Plan Exploration and Feedback-Driven Refinement. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1319–1331.

[52] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1592–1604. doi:10.1145/3650212.3680384

[53] Yang Zhang, Rui Zhang, Jiaming Guo, Lei Huang, Di Huang, Yunpu Zhao, Shuyao Cheng, Pengwei Jin, Chongxiao Li, Zidong Du, Xing Hu, Qi Guo, and Yunji Chen. 2025. QiMeng-SALV: Signal-Aware Learning for Verilog Code Generation. In *Advances in Neural Information Processing Systems*.

[54] Yang Zhao, Di Huang, Chongxiao Li, Pengwei Jin, Ziyuan Nan, Tianyun Ma, Lei Qi, Yansong Pan, Zhenxing Zhang, Rui Zhang, Xishan Zhang, Zidong Du, Qi Guo, Xing Hu, and Yunji Chen. 2024. CodeV: Empowering LLMs for Verilog Generation through Multi-Level Summarization. *arXiv preprint arXiv:2407.10424* (2024).